How Much Does Optimizer Maturity Matter? An Empirical Comparison of MySQL and Microsoft SQL Server Query Optimization Under the TPC-H Benchmark

Team Members: [Add team member names]
Master of Applied Computing (MAC) Program
School of Computer Science, University of Windsor
COMP.8157 – Advanced Database Topics, Fall 2026

***Abstract*—MySQL and Microsoft SQL Server are both full client-server relational database management systems (RDBMSs) with cost-based query optimizers, yet the two optimizers have historically differed substantially in sophistication: MySQL relied solely on nested-loop and block-nested-loop join algorithms until hash joins were introduced in MySQL 8.0.18 (2019), and its optimizer statistics remain simpler than SQL Server's histogram-driven cardinality estimator. This proposal describes a controlled, reproducible experimental study that loads the TPC-H benchmark schema and dataset into both MySQL and SQL Server at multiple scale factors, executes an identical query workload against each, and analyzes the resulting execution plans, join-order choices, and cardinality-estimation accuracy. Rather than proposing a new optimization algorithm, this project contributes an empirical, decision-oriented account of exactly where and why MySQL's optimizer diverges from SQL Server's, testing whether MySQL's post-2019 modernization has closed the practical performance gap between the two engines or whether meaningful differences remain.**

I. Introduction and Motivation

Query optimization has been a central problem in database systems research for nearly five decades, and the architecture of most modern optimizers still traces back to the cost-based, statistics-driven approach introduced in IBM's System R project [1]. Industrial RDBMSs have built increasingly sophisticated cost models, cardinality estimators, and join-enumeration strategies on top of that foundation, but they have not all done so at the same pace [2]. Microsoft SQL Server has offered a mature, histogram-based cost-based optimizer with a wide range of physical join operators for decades. MySQL, by contrast, relied exclusively on nested-loop-style join execution until hash joins were finally introduced in MySQL 8.0.18, released in 2019, and its optimizer statistics remain sampling-based and comparatively coarse. Both engines are, today, cost-based RDBMSs in the formal sense, yet they occupy noticeably different points on the optimizer-sophistication spectrum, and MySQL powers an enormous share of production web and application workloads where this gap has direct practical consequences.

This shift creates a practical problem that is not yet well documented empirically: application developers frequently choose MySQL for its operational familiarity and open-source licensing without a clear, quantified understanding of when its comparatively younger cost-based optimizer will produce meaningfully worse execution plans than a mature optimizer such as SQL Server's. Existing literature evaluates cost-based optimizers largely in isolation or compares engines of already-comparable sophistication, most influentially through the Join Order Benchmark (JOB) study of Leis et al., which found that cardinality-estimation errors – not the cost model or the enumeration strategy – are the dominant cause of poor query plans across several commercial and open-source cost-based optimizers [3]. A decade later, the same authors revisited this finding and confirmed that cardinality estimation remains the central open problem in query optimization, even as learned and AI-based approaches have entered the field [4]. However, this line of work does not directly test whether a widely deployed, historically simpler optimizer such as MySQL's has converged with a mature commercial optimizer such as SQL Server's after MySQL's 2019 join-algorithm modernization. This problem is timely and important because the answer directly informs a decision that application architects make routinely: whether MySQL's modernized optimizer is now good enough for analytical-style workloads, or whether the residual sophistication gap still justifies choosing SQL Server.

II. Problem Statement

The database problem this project addresses is characterizing, empirically and reproducibly, the performance and plan-quality gap between two cost-based query optimizers of different historical maturity – MySQL's optimizer and Microsoft SQL Server's Query Processor – under identical relational schemas, identical data, and an identical, standardized query workload. Concretely, the project asks three questions. First, at what data scale, measured using the TPC-H benchmark's scale factor, does MySQL's optimizer begin to select measurably worse join orders and access paths than SQL Server's, if at all? Second, which query shapes – multi-way joins, correlated subqueries, or aggregations over large scan ranges – are most responsible for any observed divergence? Third, how

much of the gap, where one exists, is attributable specifically to cardinality-estimation error rather than to differences in join-algorithm selection or plan-enumeration exhaustiveness, given that MySQL's hash-join support is comparatively recent. Answering these questions requires an experimental design, not a purely theoretical one, since neither engine publishes a directly comparable cost model that would allow the gap to be derived analytically.

III. Related Work

Cost-based query optimization was first formalized by Selinger et al. in the System R project, which introduced the idea of enumerating join orders and access paths and selecting the cheapest one using estimated I/O and CPU cost [1]. This architecture remains, in broad strokes, the basis for both MySQL's and SQL Server's optimizers today, and this project relies on that foundational cost-based methodology as the shared baseline against which the two engines' relative maturity is measured. Chaudhuri's survey of query optimization in relational systems frames the problem as three coupled subtasks – search-space definition, cost estimation, and plan enumeration – and this project's experimental design is structured explicitly around isolating which of these three subtasks contributes most to any MySQL/SQL Server gap, rather than treating each optimizer as a black box [2].

The most directly relevant empirical precedent is the Join Order Benchmark (JOB) study by Leis et al., which experimentally disentangled the effects of cardinality estimation, cost modeling, and plan enumeration on real-world multi-join queries and found that cardinality-estimation error, not the cost model, dominates plan quality across several commercial and open-source optimizers [3]. This project extends that methodology in a different direction: rather than comparing engines already assumed to be of comparable sophistication, it deliberately selects two engines with a documented historical maturity gap and tests whether that gap has narrowed since MySQL's 2019 hash-join introduction, using the standardized TPC-H workload rather than JOB's IMDb-derived queries. The 2025 retrospective by the same authors confirms that cardinality-estimation error has persisted as the dominant source of poor plans for a decade despite the rise of learned approaches, reinforcing that it remains an open, measurable phenomenon worth quantifying in this specific, widely deployed engine pairing [4].

Closer to the SQL Server side specifically, Lee et al. conducted a sensitivity analysis of Microsoft SQL Server's own optimizer, selectively injecting graded cardinality-estimation errors into real, complex queries (aggregates, outer joins, subqueries) to measure their effect on plan quality, and evaluated runtime mitigations such as bitmap filtering and adaptive joins [5]. This project adopts a similar sensitivity-style diagnostic methodology but applies it comparatively across engines rather than by injecting synthetic errors into one: it compares the naturally occurring estimation errors that arise from SQL Server's histogram-based statistics against MySQL's sampling-based InnoDB persistent statistics, under the same TPC-H queries used by Lee et al.'s error-injection framework as a methodological reference.

On the learned-optimizer side, Kipf et al. showed that deep-learning-based cardinality estimators can substantially outperform traditional histogram- and sampling-based estimators on correlated joins, which are exactly the query shapes this project expects to expose the largest MySQL/SQL Server gap [6]. Marcus et al.'s Neo system goes further, replacing the cardinality estimator, cost model, and plan enumerator entirely with a learned model bootstrapped from an existing optimizer such as PostgreSQL's, and reports performance competitive with commercial cost-based optimizers [7]. Rather than adopting a learned approach directly, this project treats Neo and related learned-optimizer work as motivation for why the classic three-component decomposition (cardinality estimation, cost model, plan enumeration) is still the right lens for diagnosing where a comparatively younger optimizer such as MySQL's falls short, since any future learned replacement would need exactly this kind of empirical diagnosis first. Finally, Tian's recent industrial-perspective survey argues that production query optimizers are moving from monolithic, single-engine designs toward more composable, feedback-driven architectures, and explicitly identifies robustness to estimation error as a first-class design goal industry-wide [8]. This project's findings are intended to speak directly to that trend by quantifying, for one concrete and widely deployed engine pair, how large the remaining robustness gap currently is. MySQL's own reference documentation for hash join support corroborates the historical starting point for this comparison, confirming that nested-loop-style joins were the only execution strategy available prior to version 8.0.18 [9].

Taken together, this body of work establishes both the classic cost-based architecture this project measures against ([1], [2]) and the modern empirical methodology for diagnosing optimizer weaknesses ([3], [4], [5]), while learned-optimizer research ([6], [7]) and industrial trend analysis ([8]) motivate why this diagnostic gap matters going forward. No existing study applies this diagnostic methodology to the specific, practically important question of whether MySQL's post-2019 optimizer modernization has closed its historical maturity gap with a mature commercial engine like SQL Server under a shared standardized benchmark, which is the gap this project fills.

IV. Novelty and Contributions

The novelty of this project is empirical and comparative rather than algorithmic: it does not propose a new cardinality estimator, cost model, or join-enumeration algorithm. Instead, it contributes a reproducible, standardized-benchmark-based measurement of the performance and plan-quality gap between two production cost-based optimizers with a documented historical maturity difference, broken down by query shape and data scale, together with an attribution of that gap to cardinality estimation specifically versus join-algorithm selection and plan-enumeration exhaustiveness. The assumption this project breaks is the implicit one, common in both industry practice

and casual engine comparisons, that once two engines both implement cost-based optimization with a full set of physical join operators, their plan quality has effectively converged. This project instead argues, and will demonstrate empirically, that MySQL's 2019 adoption of hash joins closed only one specific gap in a multi-component optimizer architecture, while other components – particularly cardinality estimation and the exhaustiveness of join-order search – may remain meaningfully behind SQL Server's, in ways that are only visible once both engines are measured under an identical, standardized benchmark rather than compared feature-by-feature. The resulting contribution is a concrete, reproducible decision framework: a scale-factor threshold and a set of query-shape heuristics that architects can use to decide when MySQL's optimizer is now adequate for analytical workloads and when the added cost of a mature engine like SQL Server remains justified.

V. System Design and Algorithm Overview

The experimental system consists of three components. The first is a data and schema loader that materializes the TPC-H benchmark schema and generated data into both MySQL and SQL Server at several scale factors (SF = 1, SF = 5, and SF = 10), using TPC-H's standard `dbgen` data generator to guarantee that both engines operate over byte-identical data. The second component is a query execution and plan-capture harness: a driver script issues the full TPC-H query set (22 queries spanning simple scans, multi-way joins, correlated subqueries, and aggregations) against both engines, capturing the actual execution plan, estimated versus actual row counts at each plan node, and end-to-end latency. For SQL Server, plan capture uses SQL Server Management Studio's Actual Execution Plan XML output, obtained programmatically via `SET STATISTICS XML ON`; for MySQL, plan capture uses `EXPLAIN FORMAT=JSON` together with `EXPLAIN ANALYZE`, which reports both estimated and actual row counts per plan node in MySQL 8.0.18 and later. The third component is an analysis layer that aligns the two engines' plans query-by-query, classifies each query by shape (join count, presence of correlated subqueries, aggregation type, join algorithm chosen), and computes cardinality-estimation error (the ratio of estimated to actual row count at each join node) as the primary diagnostic metric, following the sensitivity-analysis methodology of Lee et al. [5] and the estimation-error framework of Leis et al. [3]. Because both engines are full client-server RDBMSs with a comparable buffer-pool/cache architecture, this design avoids the storage-model confound that would arise from comparing an embedded engine against a server engine, and lets the analysis attribute observed differences specifically to optimizer decisions – cardinality estimation, join-algorithm choice, and plan enumeration – rather than to unrelated architectural factors.

VI. Implementation Plan

The project will be implemented primarily in Python, using the TPC-H `dbgen`/`qgen` toolkit (C, compiled locally) to generate the benchmark data and query templates at each scale factor. Data loading and query execution will be orchestrated through `pyodbc` for SQL Server and MySQL Connector/Python for MySQL, with SQL Server Management Studio (SSMS) and MySQL Workbench used interactively to visually validate execution plans and cross-check the programmatically captured plan output. Plan XML (SQL Server) and JSON-formatted `EXPLAIN`/`EXPLAIN ANALYZE` output (MySQL) will be parsed into a common intermediate representation (join order, join algorithm, access method, and per-node cardinality estimate) so that the two engines' plans can be compared node-by-node despite their different native plan formats. Both engines will be run as local server instances on the same machine to avoid introducing a network-latency confound between the two comparisons. All experiment code, the generated TPC-H datasets (or the scripts used to regenerate them, since raw TPC-H data is large), the resulting plan captures, and the analysis notebooks will be stored in a public GitHub repository, as required by the course. Results will be summarized as a set of latency-versus-scale-factor charts per query, a table of cardinality-estimation error and join-algorithm choice by query shape, and a short written decision framework summarizing the findings.

VII. Risks and Mitigation

The primary risk is that MySQL's 2019 hash-join modernization may have closed the practical gap with SQL Server entirely at the scale factors tested, leaving no meaningful divergence to report; this is mitigated by testing at SF = 1, 5, and 10 and, if necessary, extending to SF = 20, and by explicitly treating a null result (optimizer convergence) as a valid and reportable finding, consistent with the course's stated grading philosophy that honestly reported results are valued over dramatic ones. A second risk is that SQL Server's execution plan XML and MySQL's JSON-formatted EXPLAIN output use fundamentally different terminology and structure, which could make node-by-node comparison unreliable; this is mitigated by building the common intermediate representation early in the project timeline and validating it manually against a small number of queries before automating the full comparison. A third risk is environment access, since SQL Server requires either a licensed instance or SQL Server Developer Edition; this is mitigated by using SQL Server Developer Edition, which is free for non-production development and sufficient for this benchmark, alongside the community edition of MySQL. Finally, there is a risk that the team's available compute resources may not comfortably run both server engines and generate TPC-H data at SF = 10 locally; this is mitigated by using cloud-hosted trial instances or scaling down to SF = 5 as a fallback while still reporting the smaller-scale results honestly.

VIII. Timeline and Milestones

By October 11, the team will have TPC-H data generated at SF = 1 and SF = 5 and loaded into both MySQL and SQL Server, with schema and data integrity verified in both engines.

By October 18, the query execution and plan-capture harness will be complete for both engines, with all 22 TPC-H queries running successfully and producing captured plan output. By October 25, the plan-alignment and cardinality-estimation-error analysis layer will be implemented and validated against a subset of queries. By November 1, the full experiment will be run across all scale factors, with preliminary results ready ahead of the November 8 midterm project review. Between November 8 and November 29, the team will extend the analysis with any additional scale factors or query subsets needed to sharpen the findings, finalize the GitHub repository and accompanying documentation, and prepare the final report and presentation materials ahead of the November 29 demo and December 1 final report deadline.

REFERENCES

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD '79)*, 1979, pp. 23–34.

[2] S. Chaudhuri, "An overview of query optimization in relational systems," in *Proc. ACM SIGACT-SIGMOD-SIGART Symp. Principles of Database Systems (PODS '98)*, 1998, pp. 34–43.

[3] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015.

[4] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "Still asking: How good are query optimizers, really?" *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5531–5536, 2025.

[5] K. Lee, A. Dutt, V. Narasayya, and S. Chaudhuri, "Analyzing the impact of cardinality estimation on execution plans in Microsoft SQL Server," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 2871–2883, 2023.

[6] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper, "Learned cardinalities: Estimating correlated joins with deep learning," *arXiv preprint arXiv:1809.00677*, 2018.

[7] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1705–1718, 2019.

[8] Y. Tian, "Query optimization in the wild: Realities and trends," *arXiv preprint arXiv:2510.20082*, 2025.

[9] Oracle Corp., "MySQL 8.0 Reference Manual: Hash Join Optimization," https://dev.mysql.com/doc/refman/en/hash-joins.html, accessed 2026.